TUGAS LAPRAK PERTEMUAN 6 PEMROGRAMAN WEB PRAKTIK

KONFIGURASI REST API

Disusun Oleh:

5240411083 - KHALID AZIZ

UNIVERSITAS TEKNOLOGI YOGYAKARTA

FAKULTAS SAINS DAN TEKNOLOGI

2026/2027

PENDAHULUAN

1.1 Latar Belakang

REST API merupakan teknologi yang digunakan untuk menghubungkan aplikasi melalui pertukaran data menggunakan protokol HTTP. Dalam praktikum ini, Flask digunakan sebagai framework Python untuk membuat REST API dengan respons berupa data JSON. Praktikum berfokus pada pembuatan lima endpoint, penggunaan metode HTTP, serta pengujian hasil respons API. Melalui kegiatan ini, mahasiswa diharapkan dapat memahami perbedaan antara respons tampilan web dan respons data dari API.

1.2 Tujuan Praktikum

Tujuan praktikum ini adalah memahami konsep dasar REST API, mempelajari konfigurasi Flask menggunakan Virtual Environment, membuat lima endpoint, serta menguji dan memahami hasil respons API dalam format JSON.

IMPLEMENTASI PROJECT REST API

1. Membuat, Menginstal, dan Mengaktifkan Venv, Flask dan Requirements

A. Persiapan Project

Praktikum diawali dengan membuat folder proyek bernama student_task_api menggunakan Visual Studio Code. Proyek ini dibuat untuk membangun REST API Manajemen Tugas Mahasiswa menggunakan bahasa pemrograman Python dan framework Flask.

B. Konfigurasi Virtual Environment

Virtual Environment dibuat menggunakan perintah python -m venv .venv. Setelah itu, lingkungan virtual diaktifkan melalui PowerShell menggunakan perintah .\.venv\Scripts\Activate.ps1.

Penggunaan Virtual Environment bertujuan untuk memisahkan library yang digunakan dalam proyek agar tidak bercampur dengan instalasi Python secara global.

C. Instalasi Library Flask

Setelah Virtual Environment aktif, dilakukan instalasi Flask menggunakan perintah python -m pip install flask. Flask merupakan framework Python yang digunakan untuk membangun aplikasi REST API.

Selanjutnya, perintah python -m pip freeze > requirements.txt digunakan untuk menyimpan daftar library beserta versinya agar konfigurasi proyek dapat digunakan kembali.

```
PS C:\Users\khali\OneDrive\Dokumen\PWP2_IV\student_task_api> Get-
ChildItem -Force


    Directory:
C:\Users\khali\OneDrive\Dokumen\PWP2_IV\student_task_api


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
dar--l         11/5/2026   2:27 AM                venv


PS C:\Users\khali\OneDrive\Dokumen\PWP2_IV\student_task_api> Test-
Path .\.venv\Scripts\Activate.ps1
False
PS C:\Users\khali\OneDrive\Dokumen\PWP2_IV\student_task_api>
python -m venv .venv
PS C:\Users\khali\OneDrive\Dokumen\PWP2_IV\student_task_api> Test-
Path .\.venv\Scripts\Activate.ps1
```

```
True
PS C:\Users\khali\OneDrive\Dokumen\PWP2_IV\student_task_api>
.\.venv\Scripts\Activate.ps1
(.venv) PS
C:\Users\khali\OneDrive\Dokumen\PWP2_IV\student_task_api> python -
m pip install flask
Collecting flask
  Using cached flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from flask)
  Using cached blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask)
  Using cached click-8.5.0-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask)
  Using cached itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9
kB)
Collecting jinja2>=3.1.2 (from flask)
  Using cached jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from flask)
  Using cached markupsafe-3.0.4-cp313-cp313-win_amd64.whl.metadata
(2.8 kB)
Collecting werkzeug>=3.1.0 (from flask)
  Using cached werkzeug-3.1.9-py3-none-any.whl.metadata (4.1 kB)
Using cached flask-3.1.3-py3-none-any.whl (103 kB)
Using cached blinker-1.9.0-py3-none-any.whl (8.5 kB)
Using cached click-8.5.0-py3-none-any.whl (125 kB)
Using cached itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Using cached jinja2-3.1.6-py3-none-any.whl (134 kB)
Using cached markupsafe-3.0.4-cp313-cp313-win_amd64.whl (14 kB)
Using cached werkzeug-3.1.9-py3-none-any.whl (228 kB)
Installing collected packages: markupsafe, itsdangerous, click,
blinker, werkzeug, jinja2, flask
Successfully installed blinker-1.9.0 click-8.5.0 flask-3.1.3
itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.4 werkzeug-3.1.9

[notice] A new release of pip is available: 24.2 -> 26.3
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS
C:\Users\khali\OneDrive\Dokumen\PWP2_IV\student_task_api> python -
m pip freeze > requirements.txt
(.venv) PS
C:\Users\khali\OneDrive\Dokumen\PWP2_IV\student_task_api> pip list
Package      Version
------------ -------
blinker      1.9.0
click        8.5.0
Flask        3.1.3
itsdangerous 2.2.0
Jinja2       3.1.6
MarkupSafe   3.0.4
pip          24.2
Werkzeug     3.1.9
(.venv) PS
C:\Users\khali\OneDrive\Dokumen\PWP2_IV\student_task_api> pip list
Package      Version
------------ -------
```

```
blinker      1.9.0
click        8.5.0
Flask        3.1.3
itsdangerous 2.2.0
Jinja2       3.1.6
MarkupSafe   3.0.4
pip          24.2
Werkzeug     3.1.9
(.venv) PS
C:\Users\khali\OneDrive\Dokumen\PWP2_IV\student_task_api>
```

2. Membuat REST API

Pembuatan program dilakukan dengan membuat file app.py yang berisi konfigurasi Flask, data tugas mahasiswa, serta lima endpoint dengan fungsi yang berbeda. Data tugas disimpan sementara dalam bentuk list dan dictionary Python. Setiap data memiliki atribut ID, nama tugas, mata kuliah, dan status penyelesaian.

A. Menjalankan Aplikasi

Aplikasi dijalankan melalui terminal VS Code menggunakan perintah python -m flask --app app run --debug. Setelah berhasil dijalankan, aplikasi dapat diakses melalui alamat http://127.0.0.1:5000 untuk melihat respons JSON dari endpoint yang tersedia.

B. Pengujian Endpoint REST API

Pengujian dilakukan untuk memastikan setiap endpoint dapat menjalankan fungsinya dengan benar. Pengujian meliputi pengambilan data, penambahan data, dan penghapusan data menggunakan metode HTTP GET, POST, dan DELETE.

```
from flask import Flask, request, jsonify

app = Flask(__name__)

# Data tugas
tasks = [
    {
        "id": 1,
        "nama": "Membuat Laporan Praktikum",
        "mata_kuliah": "Pemrograman Web",
        "status": "Belum Selesai"
    },
    {
        "id": 2,
        "nama": "Mempelajari REST API",
        "mata_kuliah": "Pemrograman Web",
        "status": "Selesai"
    }
]

# Endpoint 1: Informasi API
@app.route('/', methods=['GET'])
def home():
    return jsonify({
```

```
        "message": "Selamat datang di Student Task Management
API",
        "total_endpoint": 5
    })

# Endpoint 2: Menampilkan semua tugas
@app.route('/tasks', methods=['GET'])
def get_tasks():
    return jsonify({
        "message": "Berhasil mengambil semua data tugas",
        "dataset": tasks
    })

# Endpoint 3: Menampilkan tugas berdasarkan ID
@app.route('/tasks/<int:id>', methods=['GET'])
def get_task(id):
    task = next((t for t in tasks if t["id"] == id), None)

    if task is None:
        return jsonify({"message": "Tugas tidak ditemukan"}), 404

    return jsonify({
        "message": "Berhasil mengambil data tugas",
        "dataset": task
    })

# Endpoint 4: Menambahkan tugas
@app.route('/tasks', methods=['POST'])
def add_task():
    data = request.get_json()

    if not data or not data.get("nama") or not
data.get("mata_kuliah"):
        return jsonify({
            "message": "Nama dan mata kuliah wajib diisi"
        }), 400

    new_task = {
        "id": max([t["id"] for t in tasks], default=0) + 1,
        "nama": data["nama"],
        "mata_kuliah": data["mata_kuliah"],
        "status": data.get("status", "Belum Selesai")
    }

    tasks.append(new_task)

    return jsonify({
        "message": "Tugas berhasil ditambahkan",
        "dataset": new_task
    }), 201

# Endpoint 5: Menghapus tugas
@app.route('/tasks/<int:id>', methods=['DELETE'])
def delete_task(id):
    task = next((t for t in tasks if t["id"] == id), None)
```

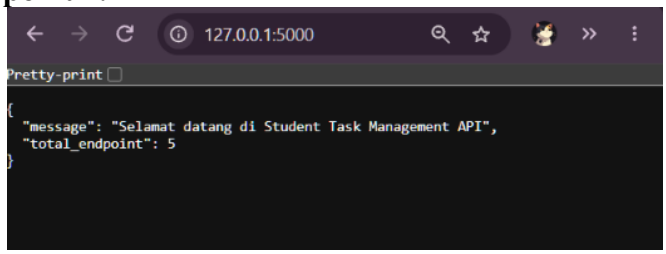
```
    if task is None:
        return jsonify({"message": "Tugas tidak ditemukan"}), 404

    tasks.remove(task)

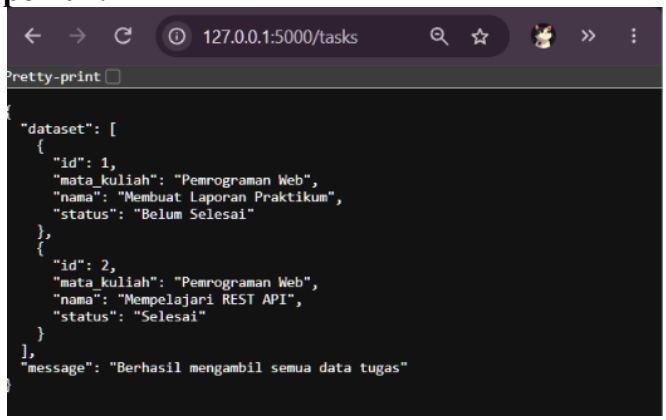
    return jsonify({
        "message": "Tugas berhasil dihapus",
        "dataset": task
    })

if __name__ == '__main__':
    app.run(debug=True)
```

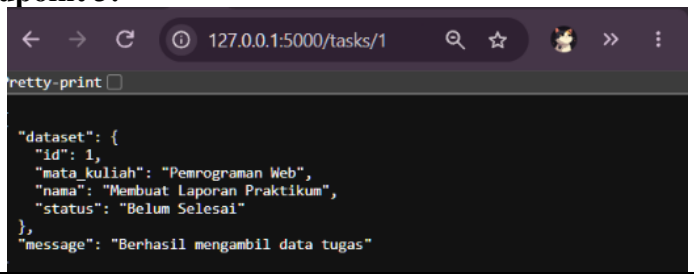
Hasil Run Uji Endpoint 1:



Hasil Run Uji Endpoint 2:



Hasil Run Uji Endpoint 3:



Endpoint 4 – Menambahkan Data Tugas (POST)

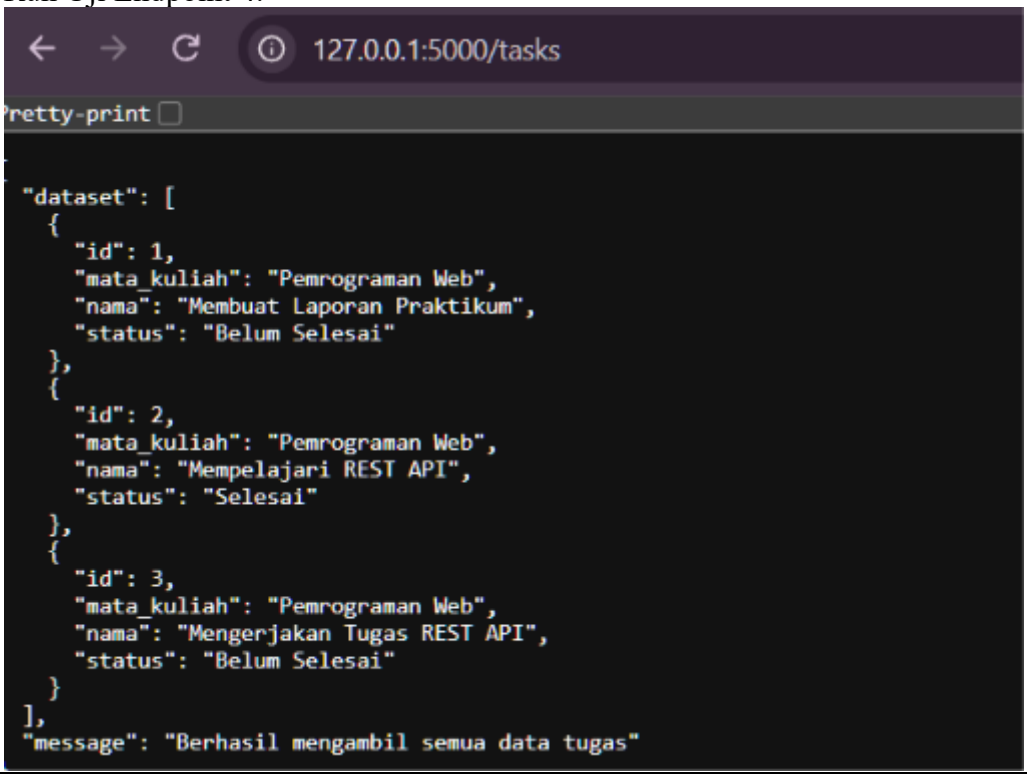
```
PS C:\Users\khali\OneDrive\Dokumen\PWP2_IV\student_task_api> (Set-
ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (&
c:\Users\khali\OneDrive\Dokumen\PWP2_IV\student_task_api\.venv\Scr
ipts\Activate.ps1)
```

```
(.venv) PS
C:\Users\khali\OneDrive\Dokumen\PWP2_IV\student_task_api> $body =
@{
>>      nama = "Mengerjakan Tugas REST API"
>>      mata_kuliah = "Pemrograman Web"
>>      status = "Belum Selesai"
>> } | ConvertTo-Json
>>
>> Invoke-RestMethod `
>>      -Uri "http://127.0.0.1:5000/tasks" `
>>      -Method POST `
>>      -ContentType "application/json" `
>>      -Body $body

dataset
-------
@{id=3; mata_kuliah=Pemrograman Web; nama=Mengerjakan Tugas REST
AP...


(.venv) PS
C:\Users\khali\OneDrive\Dokumen\PWP2_IV\student_task_api>
```

Hasil Run Uji Endpoint 4:



Endpoint 5 – Menghapus Data (DELETE)

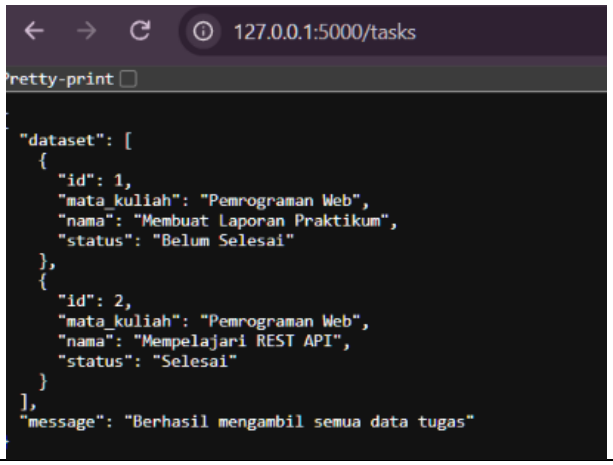
```
PS C:\Users\khali\OneDrive\Dokumen\PWP2_IV\student_task_api> (Set-
ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (&
c:\Users\khali\OneDrive\Dokumen\PWP2_IV\student_task_api\.venv\Scr
ipts\Activate.ps1)
```

```
(.venv) PS
C:\Users\khali\OneDrive\Dokumen\PWP2_IV\student_task_api> Invoke-
RestMethod `
>>      -Uri "http://127.0.0.1:5000/tasks/3" `
>>      -Method DELETE

dataset
-------
@{id=3; mata_kuliah=Pemrograman Web; nama=Mengerjakan Tugas REST
API; status=Belum Se...


(.venv) PS
C:\Users\khali\OneDrive\Dokumen\PWP2_IV\student_task_api>
```

Hasil Run Uji Endpoint 5:



Hasil Praktikum

Berdasarkan pengujian yang telah dilakukan, proyek Student Task Management API berhasil dibuat menggunakan Flask. Kelima endpoint telah diuji sesuai fungsinya, mulai dari menampilkan informasi API, mengambil data tugas, mencari tugas berdasarkan ID, menambahkan data, hingga menghapus data. Seluruh endpoint memberikan respons dalam format JSON sehingga menunjukkan penerapan konsep dasar REST API menggunakan Flask.

1. Apa itu REST API?

REST API adalah mekanisme komunikasi antar aplikasi melalui HTTP untuk mengirim dan menerima data. Dalam proyek kita, data dikirim dan diterima dalam format JSON.

2. Apa fungsi Flask?

Flask merupakan framework Python yang membantu kita membuat aplikasi web dan REST API. Dalam proyek ini Flask digunakan untuk membuat endpoint dan mengatur respons JSON.

3. Apa itu endpoint?

Endpoint adalah alamat atau URL yang digunakan untuk mengakses fungsi tertentu dari API.

4. Apa perbedaan GET, POST, dan DELETE?

Method	Kegunaan
GET	Mengambil data
POST	Menambahkan data
DELETE	Menghapus data

5. Mengapa menggunakan JSON?

Karena JSON memiliki struktur sederhana berupa pasangan key dan value sehingga mudah dibaca dan diproses oleh aplikasi.